🌐 Capa 4: Backend y Comunicación API (JSON Server y Nitro)

Ninguna aplicación moderna es estática. Necesitamos guardar canciones, leer listas y borrar registros. En este proyecto usamos una arquitectura de dos niveles para el backend.

---

1. El Motor de Datos: JSON Server

Para que los estudiantes puedan practicar sin configurar una base de datos compleja (como MySQL o MongoDB), usamos **JSON Server**.

🧠 ¿Qué es?

Es una herramienta que toma un archivo de texto (`db.json`) y lo convierte mágicamente en una **API REST profesional**.

- Permite hacer `GET` (leer), `POST` (crear), `PUT/PATCH` (actualizar) y `DELETE` (borrar).

---

2. El Puente: Capa de Servidor Nitro (`server/api`)

Aunque podríamos conectar el frontend directamente al JSON Server, Nuxt nos ofrece una capa de servidor propia llamada **Nitro**.

🧠 ¿Por qué usar Nitro como intermediario?

1. **Seguridad:** Podemos ocultar claves de API o tokens que el usuario no debe ver.
2. **Transformación:** Si el backend nos da los datos de forma "sucia", Nitro puede limpiarlos antes de enviarlos al frontend.
3. **Simulación Real:** Prepara a los estudiantes para trabajar con arquitecturas reales donde el frontend nunca toca la base de datos directamente.

💻 Ejemplo de un Endpoint en Nitro (`server/api/tracks.get.ts`):

```
export default defineEventHandler(async (event) => {
  // Pedimos los datos al JSON Server (backend real)
  const config = useRuntimeConfig()
  const tracks = await $fetch(`${config.public.apiBase}/tracks`)

  // Devolvemos los datos al Frontend
  return tracks
})
```

---

3. Consumo en el Frontend ($fetch)

Para pedirle datos a nuestra API desde Pinia o desde una página, usamos la función `$fetch` .

💻 Ejemplo de Creación (POST):

```
// En el Store (Capa 2)
async function createTrack(newTrack) {
  const response = await $fetch('/api/tracks', {
    method: 'POST',
    body: newTrack
  })
  // Actualizamos nuestra lista local para que el usuario vea el cambio al instante
  tracks.value.push(response)
}
```

---

4. El Ciclo de Vida de una Petición

1. **Usuario** hace clic en "Guardar".
2. **Componente** (Capa 1) emite un evento o llama a una acción del Store.
3. **Store** (Capa 2) usa `$fetch` para llamar a la ruta de **Nitro** (`/api/tracks`).
4. **Nitro** (Capa 4) procesa la petición y llama al **JSON Server** (Backend).
5. **JSON Server** guarda el dato en `db.json` y responde "OK".
6. La respuesta viaja de vuelta hasta el **Componente** para mostrar un mensaje de éxito.

🚀 Resumen para Estudiantes:

Término	Definición	Analogía
REST API	Estándar de comunicación.	Un menú de restaurante (pides lo que hay).
JSON Server	Base de datos simplificada.	El almacén de comida.
Nitro Server	Servidor interno de Nuxt.	El camarero (toma tu pedido y lo lleva a cocina).
$fetch	Herramienta para pedir datos.	El teléfono para llamar al restaurante.

💡 Desafío para Estudiantes:

Intenta abrir el archivo `server/db.json` después de añadir una canción. ¡Verás que tu nueva canción aparece allí escrita automáticamente!